Lecture 17: Alternative Investment Planning

Instructor: Zijie Zhou, IEDA, HKUST

November 10, 2025

Imagine you are a portfolio manager at a large endowment fund. You've decided to allocate part of your portfolio to a private equity fund.

Here's the catch: investing isn't as simple as just transferring a lump sum. Instead, you make a commitment: a promise to provide, say, $100 million over time. The fund managers won't need all that money on day one. They will 'call' your capital in chunks when they find a promising company to buy. Later, when they sell those companies, they 'distribute' money back to you.

Alternative Investment Planning

Your job is to plan these commitments. You have two main goals:

- **Hit a Target:** You want the amount of money you have actively working in the fund (the Net Asset Value or NAV) to be as close as possible to a specific target, say $50 million. You don't want to be over-invested or under-invested.
- **Be Predictable:** Your board hates surprises. They want a smooth, predictable commitment schedule. You can't commit $100 million one quarter and $10 million the next. You need to avoid wild swings.

Alternative Investment Planning

Your job is to plan these commitments. You have two main goals:

- **Hit a Target:** You want the amount of money you have actively working in the fund (the Net Asset Value or NAV) to be as close as possible to a specific target, say $50 million. You don't want to be over-invested or under-invested.
- **Be Predictable:** Your board hates surprises. They want a smooth, predictable commitment schedule. You can't commit $100 million one quarter and $10 million the next. You need to avoid wild swings.

Our challenge is to answer this question: **What should my quarterly commitment amounts be for the next several years to best achieve both of these goals simultaneously?**

The Planning Problem

Your Goal: Choose quarterly commitments $c_1, \ldots, c_T$ to a private equity fund that:

- **Tracks a target:** Keeps the invested capital (**NAV**, n_t) near a desired level n^{des}.
- **Is smooth:** Avoids sudden jumps in the commitment schedule.

We formalize this by minimizing a cost function:

$$\text{Total Cost} = \underbrace{\frac{1}{T+1} \sum_{t=1}^{T+1} \left(n_t - n^{\text{des}}\right)^2}_{\text{Tracking Error}} + \lambda \cdot \underbrace{\frac{1}{T-1} \sum_{t=1}^{T-1} (c_{t+1} - c_t)^2}_{\text{Smoothing Penalty}}$$

The Fund's Behavior: Key Variables and Dynamics

Variables We **Control**

Symbol	Description	Unit
c_t	**Commitment** we promise in quarter t	\$ Million

Variables Dictated by the **Fund's Model**

Symbol	Description	Unit
p_t	**Capital Call** (amount the fund asks for)	\$ Million
d_t	**Distribution** (amount the fund returns)	\$ Million
n_t	**Net Asset Value** (capital actively invested)	\$ Million
u_t	**Uncalled Commitment** (promised but not yet called)	\$ Million

The Fund's Behavior: Key Variables and Dynamics

How The System Evolves (The Rules of the Game)

$$n_{t+1} = n_t(1 + r_t) + p_t - d_t \qquad \text{(Value of investment changes)}$$

$$u_{t+1} = u_t - p_t + c_t \qquad \text{(Uncalled amount changes)}$$

$$p_t = \gamma^{\text{call}} \cdot u_t \qquad \text{(Fund calls a \% of uncalled capital)}$$

$$d_t = \gamma^{\text{dist}} \cdot n_t \qquad \text{(Fund returns a \% of NAV)}$$

Convex Optimization Formulation

Goal: Find the optimal commitment schedule $\boldsymbol{c} = (c_1, \ldots, c_T)$.

$$\min_{c_t, n_t, u_t, p_t, d_t} \quad \underbrace{\frac{1}{T+1} \sum_{t=1}^{T+1} \left(n_t - n^{\text{des}}\right)^2}_{\text{Tracking Error}} + \lambda \cdot \underbrace{\frac{1}{T-1} \sum_{t=1}^{T-1} (c_{t+1} - c_t)^2}_{\text{Smoothing Penalty}}$$

subject to $\quad$ for $t = 0, 1, \ldots, T-1$:

$$n_{t+1} = n_t(1 + r_t) + p_t - d_t, \quad n_0 = 0$$
$$u_{t+1} = u_t - p_t + c_t, \quad u_0 = 0$$

$$p_t = \gamma^{\text{call}} \cdot u_t$$
$$d_t = \gamma^{\text{dist}} \cdot n_t$$

$$c_t \geq 0, \quad n_t \geq 0, \quad u_t \geq 0, \quad p_t \geq 0, \quad d_t \geq 0$$
$$c_t \leq c^{\max}$$
$$u_t \leq u^{\max}$$

Implementation

```python
# --- ALTERNATIVE INVESTMENT PLANNING - OPTIMIZATION SOLUTION ---
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt

# --- Define the Optimization Variables ---
n = cp.Variable(T+1, nonneg=True)  # NAV
u = cp.Variable(T+1, nonneg=True)  # Uncalled commitments
p = cp.Variable(T, nonneg=True)    # Capital calls
d = cp.Variable(T, nonneg=True)    # Distributions
c = cp.Variable(T, nonneg=True)    # Commitments (THIS IS WHAT WE CONTROL!)

# --- Define the Optimization Problem ---
# Objective: Minimize Tracking Error + Smoothing Penalty
tracking_error = (1/(T+1)) * cp.sum_squares(n - n_des)
smoothing_penalty = lambda_param * (1/(T-1)) * cp.sum_squares(cp.diff(c))
objective = cp.Minimize(tracking_error + smoothing_penalty)

# Constraints
constraints = []
# 1. Initial conditions
constraints += [n[0] == 0, u[0] == 0]
# 2. Maximum limits
constraints += [c <= c_max, u <= u_max]
# 3. Dynamics for each time period t
for t in range(T):
    constraints += [n[t+1] == (1 + r) * n[t] + p[t] - d[t]]
    constraints += [u[t+1] == u[t] - p[t] + c[t]]
    # 4. Fund's behavior model
    constraints += [p[t] == gamma_call * u[t]]
    constraints += [d[t] == gamma_dist * n[t]]

# --- Solve the Problem ---
prob = cp.Problem(objective, constraints)
prob.solve(solver=cp.ECOS, verbose=False) # ECOS is a good solver for this convex problem

# Check if solution was found
if prob.status not in ["infeasible", "unbounded"]:
    print(f"Optimization successful! Total Cost: {prob.value:.2f}")
    # Get the optimal values
    c_opt = c.value
    n_opt = n.value
    u_opt = u.value
```

Implementation

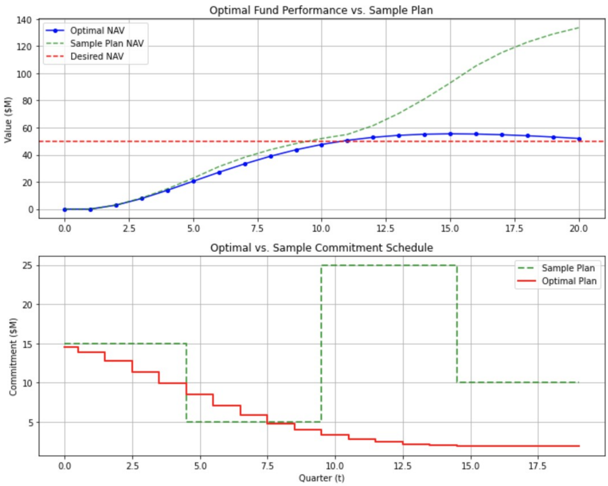
```python
# --- Calculate the Cost of the Optimal Plan ---
# .item() is used to get the scalar value from a numpy array
tracking_error_opt = np.mean((n_opt - n_des)**2).item() # FIX: Use .item()
smoothing_penalty_opt = (lambda_param * np.mean(np.diff(c_opt)**2)).item() # FIX: Use .item()
total_cost_opt = tracking_error_opt + smoothing_penalty_opt

# --- Re-run the sample simulation for comparison ---
# (This code is copied from the initial dataset cell)
c_sample = np.zeros(T)
c_sample[:5] = 15
c_sample[5:10] = 5
c_sample[10:15] = 25
c_sample[15:] = 10

n_sample = np.zeros(T+1)
u_sample = np.zeros(T+1)
p_sample = np.zeros(T)
d_sample = np.zeros(T)

for t in range(T):
    p_sample[t] = gamma_call * u_sample[t]
    d_sample[t] = gamma_dist * n_sample[t]
    n_sample[t+1] = n_sample[t] * (1 + r) + p_sample[t] - d_sample[t]
    u_sample[t+1] = u_sample[t] - p_sample[t] + c_sample[t]

# Calculate the cost of the sample plan
tracking_error_sample = np.mean((n_sample - n_des)**2).item() # FIX: Use .item()
smoothing_penalty_sample = (lambda_param * np.mean(np.diff(c_sample)**2)).item() # FIX: Use .item()
total_cost_sample = tracking_error_sample + smoothing_penalty_sample
```

Implementation

```python
# --- Plot the Optimal Solution ---
time_periods = np.arange(T+1)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

# Plot 1: Optimal NAV vs. Desired NAV
ax1.plot(time_periods, n_opt, 'b-', label='Optimal NAV', marker='o', markersize=4)
ax1.plot(time_periods, n_sample, 'g--', label='Sample Plan NAV', alpha=0.7)
ax1.axhline(y=n_des, color='r', linestyle='--', label='Desired NAV')
ax1.set_ylabel('Value ($M)')
ax1.set_title('Optimal Fund Performance vs. Sample Plan')
ax1.legend()
ax1.grid(True)

# Plot 2: Optimal Commitment Schedule vs. Sample
ax2.step(np.arange(T), c_sample, 'g--', where='mid', label='Sample Plan', linewidth=2, alpha=0.7)
ax2.step(np.arange(T), c_opt, 'r-', where='mid', label='Optimal Plan', linewidth=2)
ax2.set_xlabel('Quarter (t)')
ax2.set_ylabel('Commitment ($M)')
ax2.set_title('Optimal vs. Sample Commitment Schedule')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

# --- Print the Results ---
print(f"\nCost Breakdown:")
print(f"{'':<25} {'Optimal Plan':<15} {'Sample Plan':<15}")
print(f"{'Tracking Error:':<25} {tracking_error_opt:<15.2f} {tracking_error_sample:<15.2f}")
print(f"{'Smoothing Penalty:':<25} {smoothing_penalty_opt:<15.2f} {smoothing_penalty_sample:<15.2f}")
print(f"{'Total Cost:':<25} {total_cost_opt:<15.2f} {total_cost_sample:<15.2f}")
```

Implementation

```
Optimization successful! Total Cost: 659.42
```



Cost Breakdown:

	Optimal Plan	Sample Plan
Tracking Error:	586.50	1938.57
Smoothing Penalty:	72.92	3815.79
Total Cost:	659.42	5754.36